

# Biogas Service Management App

REA Service Application

## Complete System Run Guide

Every step to install, configure Firebase, bootstrap offices and users, and run on mobile and web.

---

Includes: Flutter setup, Isar codegen, Firebase, Firestore rules/indexes, role bootstrap, offline sync, troubleshooting, production checklist.

GitHub: [github.com/munashekamuche/biogas\\_app](https://github.com/munashekamuche/biogas_app)

## Table of Contents

1. System Overview
2. Prerequisites
3. Get the Source Code
4. Install Flutter Dependencies
5. Generate Isar Database Code
6. Firebase Project Setup
7. Register Mobile Apps in Firebase
8. Deploy Firestore Rules and Indexes
9. Assets and Branding

10. Run the Application
11. Bootstrap the System (First Run)
12. Daily Operation by Role
13. Data Model Reference
14. Offline and Sync
15. Application Forms
16. Troubleshooting
17. Production Checklist
18. Quick Command Reference
19. Support Files in Repository

## 1. System Overview

The Biogas Service Management App (branded in the UI as REA Service Application) is a Flutter-based digital service delivery platform for Grid, Solar, and Biogas solutions. One codebase supports Android, iOS, and Web.

The system serves four user roles:

- ☑ Client (mobile): registers, selects a regional office, submits service applications, tracks status.
- ☑ Staff (mobile): logs in with email, submits field reports, optionally linked to office and station.
- ☑ Admin (mobile/web): national oversight of all offices, users, applications, and reports.
- ☑ Office (web recommended): regional portal; sees only data for assigned officeId.

Backend: Firebase Authentication, Cloud Firestore, Firebase Storage. Local offline storage: Isar database on device.

## 2. Prerequisites

- ☑ Flutter SDK 3.0 or higher. Verify with: flutter doctor
- ☑ Git (to clone from GitHub)
- ☑ Firebase account and a Firebase project
- ☑ IDE: VS Code or Android Studio (recommended)
- ☑ For Android: Android Studio, SDK, emulator or physical device with USB debugging
- ☑ For iOS (macOS only): Xcode, CocoaPods, simulator or device
- ☑ For Web: Chrome browser
- ☑ Optional: Firebase CLI (npm install -g firebase-tools) for deploying rules and indexes

## 3. Get the Source Code

Repository: [https://github.com/munashekamuche/biogas\\_app.git](https://github.com/munashekamuche/biogas_app.git)

```
git clone https://github.com/munashekamuche/biogas_app.git
cd biogas_app
```

Open the project folder in your IDE. Project root contains pubspec.yaml, lib/, android/, ios/, web/, firestore.rules, firestore.indexes.json, README.md, and SETUP\_GUIDE.md.

## 4. Install Flutter Dependencies

From the project root, run:

```
flutter pub get
```

This downloads all packages listed in pubspec.yaml including provider, isar, firebase\_core, cloud\_firestore, firebase\_auth, connectivity\_plus, pdf, printing, and others.

## 5. Generate Isar Database Code

The app uses Isar for offline storage. Generated code is NOT committed to Git (.gitignore excludes \*.g.dart). You MUST run code generation after clone:

```
dart run build_runner build --delete-conflicting-outputs
```

This creates lib/models/isar\_models.g.dart from lib/models/isar\_models.dart. Re-run whenever isar\_models.dart changes.

If generation fails due to syntax errors in lib/, fix those first, then:

```
flutter clean
flutter pub get
dart run build_runner build --delete-conflicting-outputs
```

## 6. Firebase Project Setup

### 6.1 Create Firebase Project

1. Go to <https://console.firebase.google.com/>
2. Click Add project and follow the wizard.
3. Note your project ID (e.g. bgasapp).

### 6.2 Enable Firestore

1. Firebase Console > Build > Firestore Database > Create database.
2. For development, start in test mode; choose a region close to users.
3. For production, deploy firestore.rules from this repo and switch to production rules.

### 6.3 Enable Authentication

1. Firebase Console > Build > Authentication > Get started.
2. Sign-in method > Email/Password > Enable > Save.

### 6.4 Optional: Firebase Storage

Enable Storage if you plan to upload documents or images via storage\_service.dart.

## 7. Register Mobile Apps in Firebase

### 7.1 Android

Current applicationId in android/app/build.gradle.kts: com.ref.bgas\_app

1. Firebase Console > Project settings > Your apps > Add app > Android.
2. Register package name: com.ref.bgas\_app (must match applicationId exactly).
3. Download google-services.json.
4. Place file at: android/app/google-services.json (NOT android/ root).
5. The Google Services Gradle plugin is already configured in build.gradle.kts.

## 7.2 iOS (optional)

1. Firebase Console > Add app > iOS.
2. Use bundle ID from ios/Runner/Info.plist.
3. Download GoogleService-Info.plist.
4. Place at: ios/Runner/GoogleService-Info.plist.
5. Run: `cd ios && pod install` (on macOS).

## 7.3 Web (optional)

1. Firebase Console > Add app > Web.
2. Copy Firebase config; Flutter web uses the same google-services / Firebase options as configured for your platform.
3. Ensure Firebase project allows web auth domain if hosting separately.

IMPORTANT: google-services.json and GoogleService-Info.plist are gitignored. Each developer and CI machine must add them locally. Never commit secrets to public repos.

# 8. Deploy Firestore Rules and Indexes

Multi-office queries require composite indexes. Security requires deployed rules.

## 8.1 Install Firebase CLI

```
npm install -g firebase-tools
firebase login
firebase init firestore
```

Select your Firebase project. Use existing firestore.rules and firestore.indexes.json from project root when prompted.

## 8.2 Deploy

```
firebase deploy --only firestore:rules
firebase deploy --only firestore:indexes
```

Indexes in firestore.indexes.json include: users (role + officeId), applications (officeId + submittedAt), reports (officeId + submittedAt). Without indexes, office portal queries may fail with a console link to create the index.

# 9. Assets and Branding

Required asset folders (see pubspec.yaml):

- ❑ assets/logo/company\_logo.png - splash and login logo (512x512 recommended)
- ❑ assets/images/grid\_solar.png - Grid/Solar service card image
- ❑ assets/images/biogas.png - Biogas service card image
- ❑ assets/config/form\_fields.json - optional local form field definitions

After adding assets, run `flutter pub get` if you changed pubspec.yaml asset paths.

# 10. Run the Application

## 10.1 List devices

```
flutter devices
```

## 10.2 Android

```
flutter run
# or
flutter run -d <android-device-id>
```

First launch: Splash screen > Intro (once) > Login. Ensure emulator/device has Google Play if using Firebase on Android.

## 10.3 iOS (macOS)

```
flutter run -d ios
```

## 10.4 Web (Office portal & Admin)

```
flutter run -d chrome
# production build:
flutter build web
```

Output in build/web/. Host on Firebase Hosting, Netlify, or any static host. Web layout uses max width 1200px in main.dart MaterialApp.builder.

# 11. Bootstrap the System (First Run)

Follow this order before end users can use the system:

## 11.1 Create National Admin

Method A - Firebase Console (recommended for production):

1. Authentication > Users > Add user (email + password).
2. Copy the User UID.
3. Firestore > users collection > Add document with Document ID = UID.
4. Fields: id, fullName, surname, nationalId, phoneNumber, email, role: admin, createdAt (timestamp).

Method B - In-app (development only): Login screen may link to /create-admin (CreateAdminScreen). Remove this route before production.

## 11.2 Login as Admin

1. Open app > Login.
2. Enter admin email and password (role auto-detected from Firestore user document).
3. You are routed to Admin Dashboard (/admin-dashboard).

## 11.3 Create Regional Offices

1. Admin Dashboard > Offices tab.
2. Tap Add office. Enter office name and optional region.
3. Office appears in Firestore offices collection with active: true.
4. Toggle active switch to disable an office without deleting.

Clients cannot register until at least one active office exists (Register screen loads active offices).

## 11.4 Create Office Portal User (Web)

1. Admin > Offices tab > Office web login (or navigate to /add-office-portal-user).
2. Select office, enter full name, surname, email, password.
3. User is created with role: office and officeId set.
4. Share credentials with regional office staff.

## 11.5 Create Staff Accounts

1. Admin Dashboard > Staff tab > Add Staff (/add-staff).
2. Fill form including station and optional regional office.
3. Staff receives email + password; role is staff in Firestore.

## 11.6 Client Registration (Mobile)

1. User opens app > Register.
2. Select regional office from dropdown.
3. Enter name, surname, phone, password.
4. Client login uses phone-based pseudo-email: {phoneDigits}@rea.client
5. After login, client can request Grid/Solar/Biogas services and submit application forms.

# 12. Daily Operation by Role

## 12.1 Client flow

1. Login with name, surname, phone, password.
2. Home > Service Request > choose Grid/Solar or Biogas.
3. Complete application form; submit (online or offline).
4. View applications list and status; check notifications.

## 12.2 Staff flow

1. Login with email and password.
2. Staff Home > Create Report.
3. Enter report content; submit (syncs when online).
4. View own submitted reports.

## 12.3 Office portal flow (Web)

1. Login with office portal email (role: office).
2. Office Dashboard: Applications, Reports, Clients tabs (filtered by officeId).
3. Open pending application > Approve, Reject, or In progress.

## 12.4 Admin flow

1. View all applications and reports nationally.
2. Manage staff, clients, offices.
3. Export/print where implemented in admin dashboard.

# 13. Data Model Reference

Firestore collections:

- ☒ users: id, fullName, surname, nationalId, phoneNumber, email, role, officeId?, station?, createdAt
- ☒ offices: id, name, region?, active, createdAt
- ☒ applications: id, userId, officeId, serviceType, biogasType, formData, status, submittedAt, updatedAt
- ☒ reports: id, userId, officeId, staffName, station, content, submittedAt
- ☒ notifications: userId, title, message, type, read, createdAt
- ☒ form\_configs: optional dynamic form definitions per service type

## 14. Offline and Sync

When device is offline:

- ☒ Applications and reports save to Isar with needsSync = true.
- ☒ Reads fall back to local Isar data.
- ☒ When connectivity returns, SyncService pushes pending records to Firestore.

Test offline: disable WiFi/mobile data, submit application or report, re-enable network, verify in Firebase Console.

## 15. Application Forms

Forms load via FormConfigService with priority: Firestore form\_configs > assets/config/form\_fields.json > code defaults in form\_config\_service.dart.

Grid/Solar uses sectioned form; Biogas/Solar homestead vs institutional fields differ. Customize via Firestore or JSON without code changes where supported.

## 16. Troubleshooting

| Problem                     | Solution                                                             |
|-----------------------------|----------------------------------------------------------------------|
| Isar / build_runner fails   | Fix lib/ syntax errors; flutter clean; pub get; rebuild runner       |
| Firebase init error         | Verify google-services.json path and package name match              |
| User profile not found      | Create Firestore users/{uid} doc matching Auth UID                   |
| Missing Firestore index     | Deploy firestore.indexes.json or use console link                    |
| Office portal empty         | Ensure applications have officeId; deploy indexes; check rules       |
| Client cannot register      | Create at least one active office in Admin > Offices                 |
| Permission denied Firestore | Deploy and review firestore.rules for role/officeId                  |
| Web blank or layout issues  | flutter build web; check browser console; verify Firebase web config |

## 17. Production Checklist

- ☒ Remove or protect /create-admin route and CreateAdminScreen.
- ☒ Deploy production Firestore rules (not test mode).
- ☒ Deploy all composite indexes.
- ☒ Use strong passwords; never commit Firebase config secrets to public repos.
- ☒ Configure Android release signing (not debug keys).
- ☒ Enable Firebase App Check if required.
- ☒ Set up Firebase Hosting for web portal if using office users on web.
- ☒ Plan backup and audit strategy for Firestore data.

❏ Push notifications (FCM) not enabled in pubspec yet - add `firebase_messaging` when needed.

## 18. Quick Command Reference

```
flutter doctor
flutter pub get
dart run build_runner build --delete-conflicting-outputs
flutter devices
flutter run
flutter run -d chrome
flutter build web
flutter build apk
firebase deploy --only firestore:rules,firestore:indexes
flutter clean
```

## 19. Support Files in Repository

- ❏ README.md - architecture and feature overview
- ❏ SETUP\_GUIDE.md - setup steps
- ❏ ADMIN\_SETUP.md - admin account creation
- ❏ ACCOUNT\_CREATION\_GUIDE.md - admin and staff accounts
- ❏ FIREBASE\_SETUP.md - Firebase registration details
- ❏ firestore.rules - security rules template
- ❏ firestore.indexes.json - required composite indexes

Document generated for Biogas Service Management App. For updates, regenerate with: `dart run tool/generate_run_guide_pdf.dart`